The 1995 recognizer: functional components and I/O

Status: for review, 2026-11-09. These diagrams show the recognizer from the speaker's voice to a recognized phrase. Every arrow is labelled with what passes along it. They cover the 1995 system as the thesis describes it (Chapter 5, Annexes C–G) and the Python port in `src/blindman/repro1995/` that reproduces Chapter 6 (PHASE0.md). The port's software design, its modules and classes and how they relate, is in PORT_DESIGN.md.

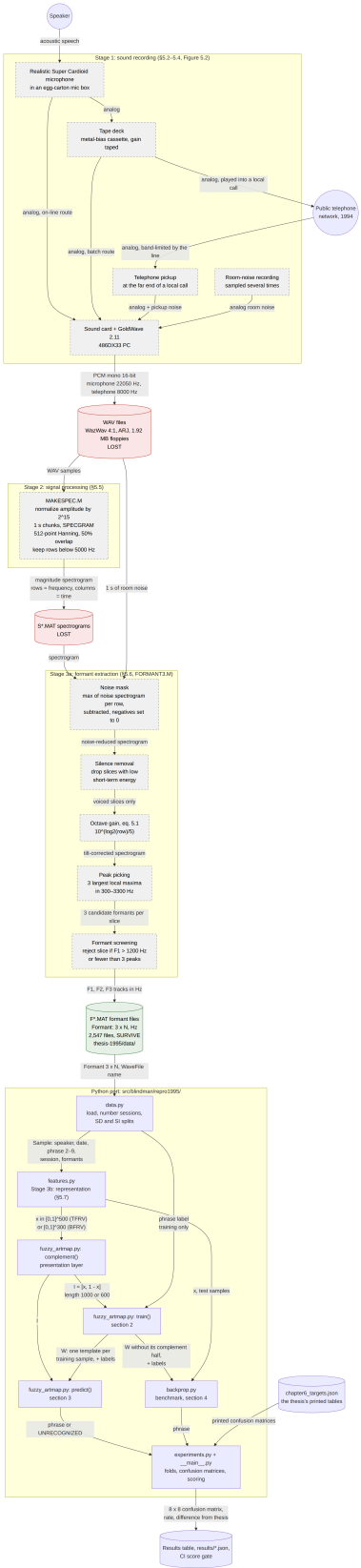
Legend:

Shape	Meaning
Circle	Outside the system: the speaker, the telephone network. This follows Figure 5.2
Rectangle, solid border	A component ported to Python, with its file name
Rectangle, dashed grey border	1995 hardware or MATLAB code that is **not** ported
Cylinder	Stored data
Red cylinder	Lost data: the WAV recordings and the spectrograms did not survive

The port starts at the formant files. The raw audio and the spectrograms are lost. The 2,547 formant files (`F*.MAT`) are the earliest data that survives, so the Python port begins there. Porting the front end, `MAKESPEC.M` and `FORMANT3.M`, is needed before new recordings can be used (STAGE2, Phase 1).

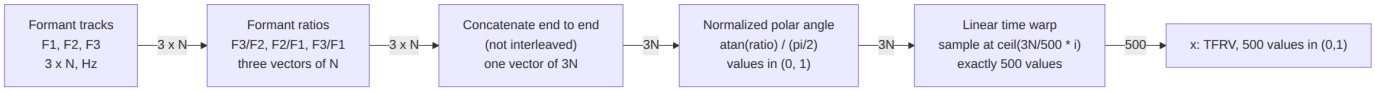
---

1. End to end


Speaker
acoustic speech
Stage 1: sound recording (§5.2–5.4, Figure 5.2)
Realistic Super Cardioid microphone in an egg-carton mic box
analog
Tape deck metal-bias cassette, gain taped
analog, played into a local call
Public telephone network, 1994
analog, on-line route
analog, batch route
analog, band-limited by the line
Telephone pickup at the far end of a local call
Room-noise recording sampled several times
analog + pickup noise
analog room noise
Sound card + GoldWave 2.15 486DX33 PC
PCM mono 16-bit microphone 22050 Hz, telephone 8000 Hz
WAV files Wkz/Wav 4:1, AR1, 1.92 MB floppies LOST
WAV samples
Stage 2: signal processing (§5.5)
MAKESPEC.M normalize amplitude by 2^15 1 s chunks, SPECGRAM 512-point Hanning, 50% overlap keep rows below 5000 Hz
magnitude spectrogram rows = frequency, columns = time
1 s of room noise
S*.MAT spectrograms LOST
spectrogram
Stage 3a: formant extraction (LPC-8, FORMANT3.M)
Noise mask max of noise spectrogram per row, subtracted, negatives set to 0
noise-reduced spectrogram
Silence removal drop slices with low short-term energy
voiced slices only
Octave gain, eq. 5.1 10*(log2(row)/5)
tilt-corrected spectrogram
Peak picking 3 largest local maxima in 300–3300 Hz
3 candidate formants per slice
Formant screening reject slice if F1 > 1200 Hz or fewer than 3 peaks
F1, F2, F3 tracks in Hz
F*.MAT formant files Formant: 3 x N, Hz 2,547 files, SURRIVE thesis-1995/data/
Formant 3 x N, WaveFile name
Python port: src/blindmanl/repro1995/
data.py load_number sessions, SD and SI splits
Sample: speaker, date, phrase 2–5, session, formants
features.py Stage 3b: representation (§5.7)
x in [0,1]^500 (TFRV) or [0,1]^300 (BFRV)
phrase label training only
fuzzy_artmap.py: complement() presentation layer
I = [x, 1 - x] length 1000 or 600
x, test samples
fuzzy_artmap.py: train() section 2
W: one template per training sample, + labels
W without its complement half, + labels
fuzzy_artmap.py: predict() section 3
backprop.py benchmark, section 4
chapter6_targets.json the thesis's printed tables
phrase or UNRECOGNIZED
phrase
printed confusion matrices
experiments.py + __main__.py folds, confusion matrices, scoring
8 x 8 confusion matrix, rate, difference from thesis
Results table, results/*.json, CI score gate

Stage 3b: from formant tracks to the network's input

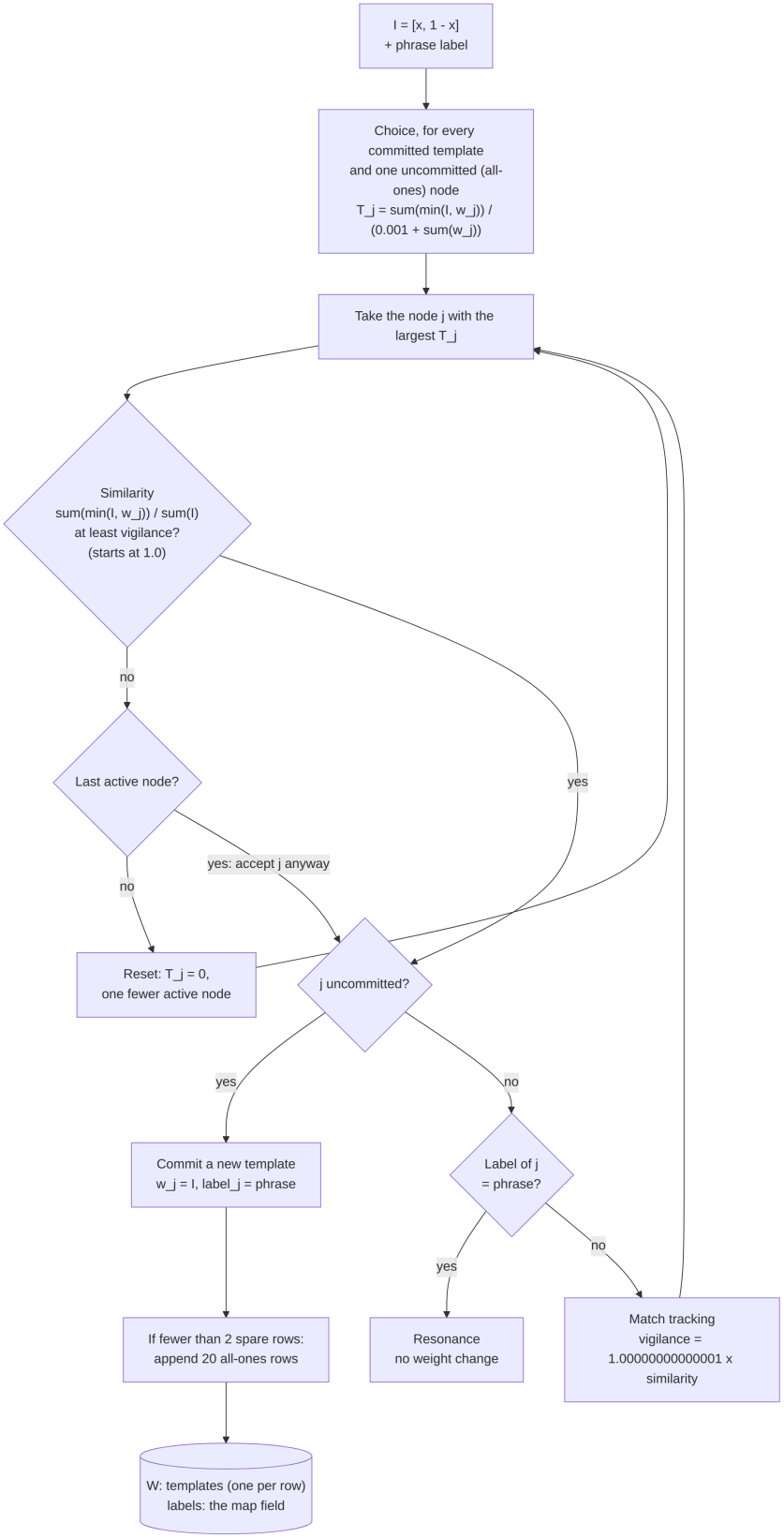
This is the normalization that makes every phrase the same length, whatever its duration (`features.py`, §5.7.3–5.7.4):



BFRV is the same with two ratios, F3/F2 and F2/F1, warped to 300. No BFRV code survives, so this path is reconstructed from §5.7.2.

---

2. Training: how the network learns a phrase



For review: training adds templates but never adjusts weights. In textbook Fuzzy ARTMAP, a node that resonates is moved towards the input, `w ← β(I ∧ w) + (1 − β)w`. FARTMAPT.M uses fast learning with no recoding (§5.8.2), so a template is set once, equal to the input that created it, and never changes. The thesis calls these templates **classification landmarks**.

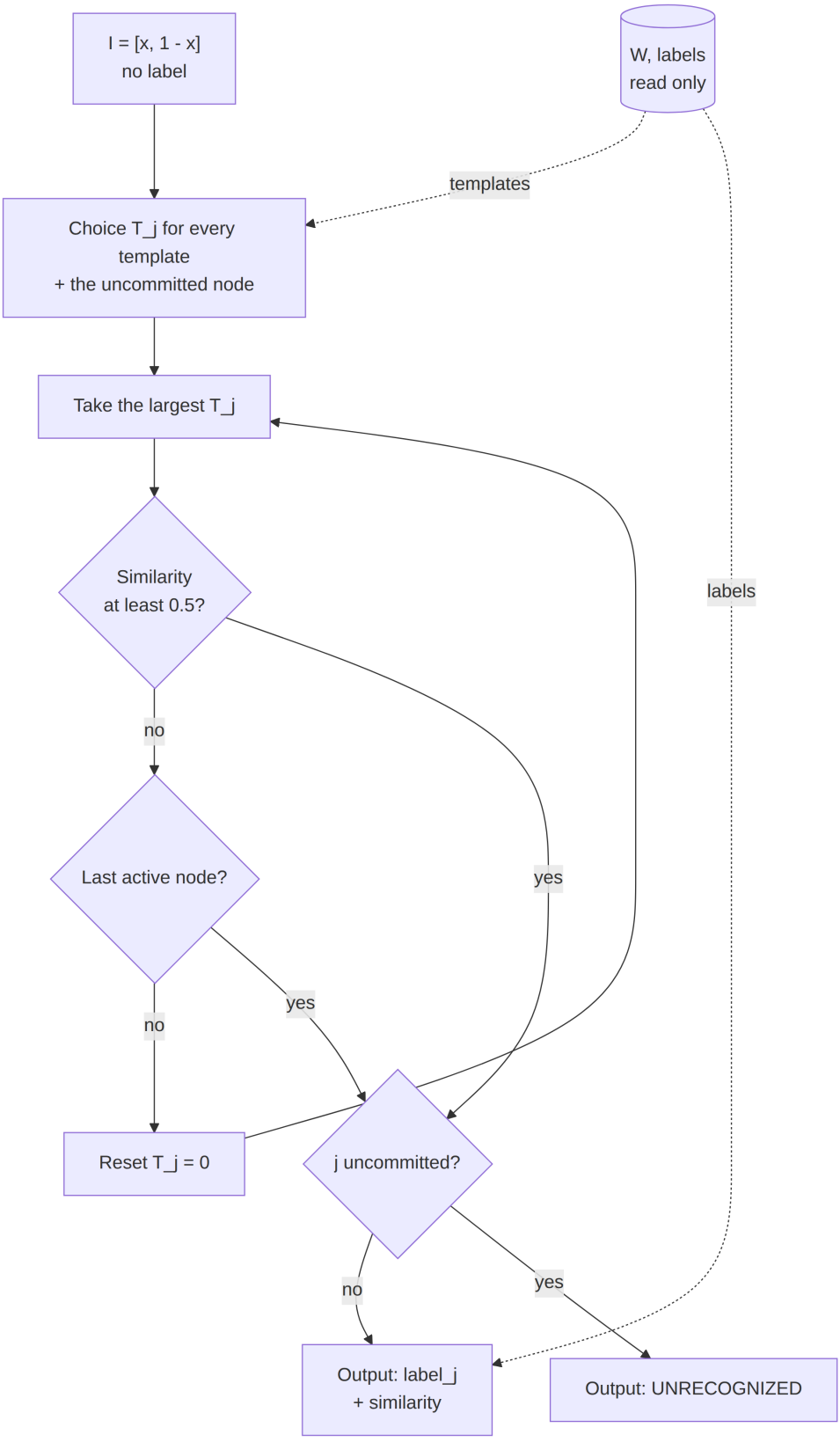
At a training vigilance of 1.0, only an identical vector passes the vigilance test against an existing template. Each training sample therefore becomes its own landmark. A check on DY's first 80 microphone samples gave 80 templates. In this configuration, Fuzzy ARTMAP training is template storage. The one place that weights are adjusted to match an output layer is the backprop benchmark (section 4).

The map field is FARTMAPT.M's simplification of ART-b (§5.8.1, Figure 5.21). It is a list of labels, one per template, and match tracking consults it.

The thesis text says that memory grows by "10 new columns" (§5.8.1). The code appends 20 rows (`W` has one template per row). The port follows the code.

---

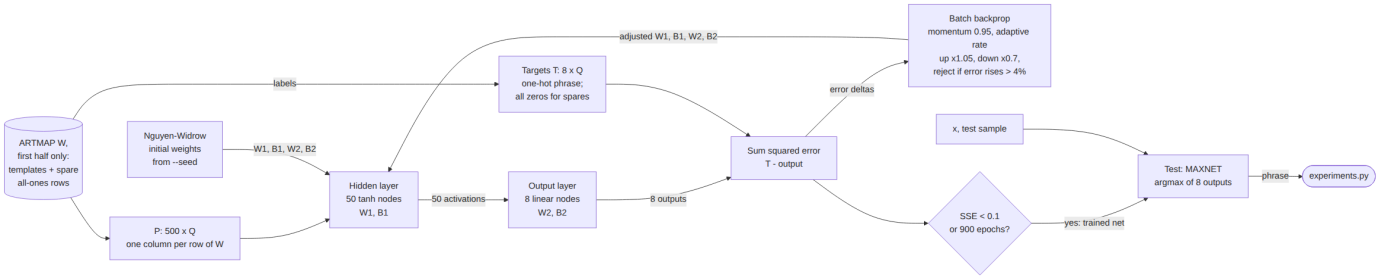
3. Testing: matching to the closest template



- **What "closest" means here:** the template with the largest choice value `T_j`, among those whose similarity passes the testing vigilance of 0.5. `T_j` measures how much of the template the input covers, `|I ∧ w_j|`, and divides it by the template's size, `|w_j|`. It is not a Euclidean distance. This is the "nearest landmark" of §5.8.3.
- **Testing never learns.** `W` is read only, and there is no match tracking.
- **The last active node is accepted** whatever its similarity. This is how FARTMAPT.M is written. The uncommitted node is the last candidate, so when nothing passes vigilance the result is UNRECOGNIZED.
- Every test compares the input against every template. The thesis notes that this cost grows with memory (§5.8.3).

---

4. The backprop benchmark



The 1995 initial weights (`bpinit.mat`) and the trained network (`bprtmapt.mat`) are lost, so the port draws Nguyen-Widrow weights from a seed. Results vary by about ±1.1 points from seed to seed (PHASE0.md, note 5).

---

5. Component I/O

Component	Input	Output	Source	Ported
Microphone, mic box	Speech, acoustic	Analog voltage	§5.4	Hardware
Tape deck	Analog	Analog: to the sound card (batch) or into a local call (telephone)	§5.4, Fig. 5.2	Hardware
Telephone network + pickup	Analog	Band-limited analog with pickup noise	§5.4, §6.2.2	Hardware
Sound card, GoldWave	Analog	WAV, PCM mono 16-bit, 22050 Hz (mic) or 8000 Hz (telephone)	§5.4, Annex A	No; audio is lost
`MAKESPEC.M`	WAV	Spectrogram `S*.MAT`, 512-point Hanning, 50% overlap, below 5000 Hz	§5.5, Annex C	No; Phase 1

Component	Input	Output	Source	Ported
`FORMANT3.M`	Spectrogram + room-noise WAV	`F*.MAT`: `Formant` 3 x N in Hz, 300–3300 Hz	§5.6, Annex D	No; Phase 1
`data.py`	`thesis-1995/data/{microphone,telephone}/`	`Sample` list; SD and SI splits	§6.2.1	Yes
`features.py`	`Formant` 3 x N	TFRV in (0,1)^500, or BFRV in (0,1)^300	§5.7, Annex E	Yes (BFRV reconstructed)
`FuzzyARTMAP.complement`	x	I = [x, 1 − x]	Annex E	Yes
`FuzzyARTMAP.train`	I + phrase	New template row in `W`, label appended	§5.8.1–5.8.2, Annex E	Yes
`FuzzyARTMAP.predict`	I	Phrase or UNRECOGNIZED, + similarity	§5.8.3, Annex E	Yes
`Backprop.fit_from_artmap`	`W`, labels, seed	W1, B1, W2, B2; epochs; SSE	Annex F	Yes (initial weights reconstructed)
`Backprop.predict`	x	Phrase	Annex G	Yes
`experiments.py`	Splits, a network, targets	Confusion matrix, rate, difference from the thesis	Chapter 6	Yes
`__main__.py`	`--seed`, `--json`	Results table; JSON with an environment block	—	New